Day 2 — AI and Data Analysis (Python)

Topics Covered

1. Introduction to NumPy
2. NumPy Arrays
3. Array Creation
4. Array Indexing and Slicing
5. Two-Dimensional Arrays
6. Array Operations
7. Basic Statistical Operations

1. Introduction to NumPy

NumPy stands for Numerical Python. It is a Python library used for numerical computation and efficient operations on arrays.

Applications:

- Mathematical calculations
- Data analysis
- Scientific computing
- Data preprocessing for Machine Learning
- Working with multidimensional arrays

2. Import NumPy

```
import numpy as np
```

`np` is the conventional alias for NumPy.

3. Creating NumPy Arrays

One-Dimensional Array

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr)
```

```
print(type(arr))
```

Output:

```
[10 20 30 40 50]
<class 'numpy.ndarray'>
```

Two-Dimensional Array

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matrix)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

A two-dimensional array contains rows and columns.

4. Array Properties

```
arr = np.array([10, 20, 30, 40, 50])

print(arr.ndim)
print(arr.shape)
print(arr.size)
print(arr.dtype)
```

Meaning:

- `ndim`: Number of dimensions.
- `shape`: Size along each dimension.
- `size`: Total number of elements.
- `dtype`: Data type of the elements.

For this array:

- Dimensions: 1
- Shape: `(5,)`
- Size: 5

5. Array Indexing

Indexing accesses an individual element. NumPy uses zero-based indexing.

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[2])
print(arr[-1])
```

Output:

```
10
30
50
```

6. Array Slicing

Slicing extracts a portion of an array.

```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4])
print(arr[:3])
print(arr[2:])
```

Output:

```
[20 30 40]
[10 20 30]
[30 40 50]
```

The ending index is excluded from the slice.

7. Array Operations

NumPy supports element-wise operations.

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
print(a - b)
print(a * b)
```

```
print(a / b)
```

Output:

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

These operations are performed element by element when the arrays have compatible shapes.

8. Mathematical and Statistical Functions

```
arr = np.array([10, 20, 30, 40, 50])

print(np.sum(arr))
print(np.mean(arr))
print(np.max(arr))
print(np.min(arr))
print(np.std(arr))
```

Purpose:

- `np.sum()`: Calculates the sum.
- `np.mean()`: Calculates the arithmetic mean.
- `np.max()`: Returns the maximum value.
- `np.min()`: Returns the minimum value.
- `np.std()`: Calculates the standard deviation.

9. Reshaping an Array

Reshaping changes an array's dimensions without changing its elements.

```
arr = np.array([1, 2, 3, 4, 5, 6])

matrix = arr.reshape(2, 3)

print(matrix)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

The requested shape must be compatible with the number of elements. Here, six elements can be arranged into two rows and three columns.

10. NumPy vs Python Lists

Feature	NumPy Array	Python List
Purpose	Numerical computing	General-purpose collection
Numerical operations	Efficient element-wise operations	Usually requires loops or other operations
Dimensions	Supports multidimensional arrays	Can contain nested lists
Data types	Typically uses a common array dtype	Can contain mixed types
Common use	Data analysis and ML	General programming

Practice Exercises

1. Create a NumPy array containing numbers from 1 to 10.
2. Print the first, third, and last elements.
3. Extract elements from index 2 to index 6.
4. Create a 3 × 3 matrix.
5. Calculate the sum and mean of an array.
6. Add two arrays element by element.
7. Reshape an array of 12 elements into 3 rows and 4 columns.

Key Learnings

- Understood the purpose of NumPy.
- Learned how to create one-dimensional and two-dimensional arrays.
- Practised array indexing and slicing.
- Learned array properties such as shape, size, and dimensions.
- Performed element-wise mathematical operations.
- Used statistical functions.
- Understood reshaping and the role of NumPy in data analysis.

Revision Questions

1. What is NumPy?
2. What is the difference between `ndim`, `shape`, and `size`?

3. What is zero-based indexing?
4. Why is the ending index excluded in a slice?
5. What does `reshape()` do?
6. How can NumPy help prepare data for Machine Learning?

1. What is NumPy?

NumPy stands for Numerical Python. It is a Python library used for efficient numerical computations, multidimensional arrays, and mathematical operations.

Applications:

- Mathematical calculations
- Statistical analysis
- Scientific computing
- Data manipulation
- Preparing numerical data for Machine Learning

Example:

import numpy as nparr = np.array([10, 20, 30])print(arr)

Output:

[10 20 30]

2. What is the difference between `ndim`, `shape`, and `size`?

These are NumPy array attributes that describe different properties of an array.

Attribute	Meaning	Example
`ndim`	Number of dimensions	`2`
`shape`	Size along each dimension	`(2, 3)`

`size`	Total number of elements	`6`

Example:

import numpy as nparr = np.array([[1, 2, 3], [4, 5, 6]])print(arr.ndim) # 2print(arr.shape) # (2, 3)print(arr.size) # 6

The array has two dimensions, two rows, three columns, and six elements.

3. What is zero-based indexing?

Zero-based indexing means that the index of the first element is `0`, not `1`.

Example:

arr = np.array([10, 20, 30, 40])print(arr[0]) # 10print(arr[1]) # 20

Here:

- Index `0` refers to `10`.
- Index `1` refers to `20`.
- Index `3` refers to `40`.

Zero-based indexing is commonly used in Python and many other programming languages.

4. Why is the ending index excluded in a slice?

In Python and NumPy, the ending index of a slice is excluded. This makes it easier to calculate the number of selected elements and combine slices without overlapping boundaries.

Example:

arr = np.array([10, 20, 30, 40, 50])print(arr[1:4])

Output:

[20 30 40]

The slice includes indices `1`, `2`, and `3`, but excludes index `4`.

The number of selected elements is generally `stop - start`, provided the step and bounds are valid.

5. What does `reshape()` do?

The `reshape()` function changes the dimensions of an array without changing its elements.

Example:

import numpy as nparr = np.array([1, 2, 3, 4, 5, 6])new_arr = arr.reshape(2, 3)print(new_arr)

Output:

[[1 2 3]

 [4 5 6]]

The original one-dimensional array has been arranged into two rows and three columns.

Important: The new shape must contain the same total number of elements as the original array. Reshaping often returns a view when possible, rather than a separate copy.

6. How can NumPy help prepare data for Machine Learning?

NumPy helps prepare numerical data for Machine Learning by providing efficient arrays and mathematical operations.

Main uses:

1. Data representation: Stores numerical data in arrays and matrices.
2. Data transformation: Reshapes arrays into the required dimensions.
3. Mathematical operations: Performs addition, multiplication, and other calculations efficiently.
4. Statistical calculations: Calculates mean, minimum, maximum, and standard deviation.
5. Normalization: Supports mathematical transformations that scale numerical features.
6. Integration: Works with libraries such as Pandas and Scikit-learn.

Example:

import numpy as npmarks = np.array([60, 70, 80, 90, 100])mean_marks = np.mean(marks)normalized_marks = marks / 100print(mean_marks)print(normalized_marks)

Output:

80.0

[0.6 0.7 0.8 0.9 1.]

Here, NumPy calculates the average and scales the marks to a range from `0` to `1`. This is a simple illustration of feature scaling; actual ML preprocessing depends on the dataset and model.